

Manual de Instalación y Configuración de Herramientas de Monitoreo de Rendimiento

Pruebas de Estrés
Clínica “Bella Mujer”

Josue Chango

Julio 2026

Resumen

Este manual documenta el proceso de instalación, configuración y validación de herramientas de monitoreo de rendimiento (Grafana, Prometheus y Datadog) aplicadas a la API de citas médicas de la Clínica “Bella Mujer”. Se realiza una prueba de carga con 500 usuarios concurrentes utilizando k6, y se comparan las métricas obtenidas entre las dos plataformas de monitoreo. Los resultados demuestran que bajo la carga especificada, la API no cumple con el SLA de $T_{95} < 200\text{ms}$, requiriendo optimizaciones en la capa de persistencia.

Palabras clave: monitoreo de rendimiento, Grafana, Prometheus, Datadog, k6, pruebas de carga, tiempo de respuesta

1 Introducción

1.1 Contexto del Problema

La Clínica “Bella Mujer” lanzará su módulo web de citas médicas. Se espera que a las 11:00 a.m. ingresen simultáneamente 500 pacientes para reservar turnos. Como ingenieros de calidad, se debe certificar que la API `/api/citas` no colapsará y cumplirá con el SLA de rendimiento:

- Tiempo de respuesta $T_{95} < 200\text{ms}$ bajo carga de 500 usuarios concurrentes.

1.2 Objetivos

1. Instalar y configurar Grafana + Prometheus para monitoreo de métricas del sistema y la API.
2. Instalar y configurar Datadog para comparar métricas.
3. Desarrollar una API REST que simule el módulo de citas médicas.
4. Ejecutar pruebas de carga con k6 (500 VUs).
5. Generar una tabla comparativa entre ambas herramientas de monitoreo.
6. Elaborar el diagrama UML de tiempos de la API.

1.3 Arquitectura del Sistema

Cuadro 1: Arquitectura del Sistema

Componente	Tecnología	Ubicación
API REST	Node.js + Express	Ubuntu VM (192.168.100.41:8080)
Monitoreo 1	Prometheus + Grafana	Ubuntu VM (Docker)
Monitoreo 2	Datadog Agent v7	Ubuntu VM (nativo)
Pruebas de carga	k6 v0.49.0	Ubuntu VM
Visualización	Navegador web	Windows Host (192.168.100.36)

2 Instalación de Grafana y Prometheus

2.1 Prerrequisitos

- Ubuntu 24.04 (VM VirtualBox con adaptador puente WiFi).
- Docker y Docker Compose instalados.
- Conexión a internet.

2.2 Configuración de Docker Compose

Se crea el archivo `docker-compose.yml` con los servicios Prometheus y Grafana:

Listing 1: `docker-compose.yml`

```
version: '3.8'

services:
  prometheus:
    image: prom/prometheus:latest
    container_name: recolector_prometheus
    ports:
      - "9090:9090"
    volumes:
      - ./prometheus.yml:/etc/prometheus/prometheus.yml
    extra_hosts:
      - "host.docker.internal:host-gateway"
    restart: unless-stopped

  grafana:
    image: grafana/grafana:latest
    container_name: panel_grafana
    ports:
```

```

    - "3000:3000"

environment:
    - GF_SECURITY_ADMIN_USER=admin
    - GF_SECURITY_ADMIN_PASSWORD=admin123

depends_on:
    - prometheus

restart: unless-stopped

```

2.3 Configuración de Prometheus

Se configura el archivo `prometheus.yml` para escanear la API, el `node_exporter` del host y Prometheus a sí mismo:

Listing 2: `prometheus.yml`

```

global:
    scrape_interval: 15s
    evaluation_interval: 15s

scrape_configs:
    - job_name: 'prometheus'
      static_configs:
        - targets: ['localhost:9090']

    - job_name: 'node-exporter'
      static_configs:
        - targets: ['host.docker.internal:9100']
        labels:
            servicio: 'sistema'

    - job_name: 'bella-mujer-api'

```

```
scrape_interval: 5s
scrape_timeout: 5s
metrics_path: '/metrics'
static_configs:
  - targets: ['host.docker.internal:8080']
    labels:
      servicio: 'api-citas'
      clinica: 'Bella_Mujer'
```

2.4 Levantamiento de la Infraestructura

Listing 3: Levantamiento de contenedores

```
docker compose up -d
```

2.5 Verificación

- Grafana: <http://192.168.100.41:3000> (credenciales: admin / admin123)
- Prometheus: <http://192.168.100.41:9090>

2.6 Configuración del Data Source en Grafana

1. Menú izquierdo → Connections → Data Sources
2. Add data source → Prometheus
3. Prometheus server URL: <http://prometheus:9090>
4. Save & test

2.7 Importación del Dashboard

1. “+” → Import
2. Ingresar ID: 1860 (Node Exporter Full)

3. Seleccionar fuente de datos Prometheus

4. Importar

3 Instalación de Datadog

3.1 Creación de Cuenta

1. Navegar a <https://www.datadoghq.com/free/>
2. Crear una cuenta gratuita (free trial de 14 días).
3. Seleccionar la región correspondiente (US5).
4. Confirmar el email de verificación.

3.2 Obtención de API Key

1. En Datadog web → Organization Settings → API Keys.
2. Copiar la API Key generada (o crear una nueva).

3.3 Instalación del Agente

Listing 4: Instalación de Datadog Agent v7

```
DD_API_KEY=<API_KEY> DD_SITE="us5.datadoghq.com" \
DD_AGENT_MAJOR_VERSION=7 \
bash -c "$(curl -L \
https://s3.amazonaws.com/dd-agent/scripts/install_script_agent7.sh)"
```

3.4 Verificación

Listing 5: Verificación del agente

```
sudo datadog-agent status
```

3.5 Configuración del Site

Si la cuenta está en una región diferente, editar `/etc/datadog-agent/datadog.yaml`:

Listing 6: Configuración de Datadog

```
api_key: <API_KEY>  
site: us5.datadoghq.com
```

Reiniciar el agente:

```
sudo systemctl restart datadog-agent
```

4 API de Citas Médicas

4.1 Descripción

La API REST simula el módulo de citas médicas de la Clínica “Bella Mujer”.

Expone los siguientes endpoints:

Cuadro 2: Endpoints de la API

Método	Ruta	Descripción
GET	/api/citas	Lista citas disponibles
POST	/api/citas	Reserva una cita
GET	/metrics	Métricas Prometheus
GET	/health	Estado del servicio

4.2 Dependencias

Listing 7: package.json

```
{  
  "dependencies": {  
    "express": "^4.18.2",  
    "prom-client": "^15.1.0",  
    "cors": "^2.8.5"  
  }  
}
```

4.3 Código del Servidor

Listing 8: server.js - API Express con métricas Prometheus

```
const express = require('express');  
const client = require('prom-client');  
const cors = require('cors');  
  
const app = express();
```

```
const PORT = 8080;

const MODE = process.env.MODE || 'initial';

app.use(cors());
app.use(express.json());

// Metricas Prometheus
const collectDefaultMetrics = client.collectDefaultMetrics;
collectDefaultMetrics({ prefix: 'bella_mujer_' });

const httpRequestDuration = new client.Histogram({
  name: 'bella_mujer_http_request_duration_seconds',
  help: 'Duracion de las peticiones HTTP en segundos',
  labelNames: ['method', 'route', 'status_code'],
  buckets: [0.01, 0.025, 0.05, 0.1, 0.15,
    0.2, 0.3, 0.5, 0.75, 1, 2, 5],
});

const httpRequestTotal = new client.Counter({
  name: 'bella_mujer_http_requests_total',
  help: 'Total de peticiones HTTP',
  labelNames: ['method', 'route', 'status_code'],
});

// Simulacion de latencia de base de datos
function simulateDBQuery() {
  if (MODE === 'initial') {
    const base = 0.08 + Math.random() * 0.15;
    const occasional = Math.random() < 0.15
```

```
    ? 0.1 + Math.random() * 0.2 : 0;
    return (base + occasional) * 1000;
  }
  return (0.005 + Math.random() * 0.03) * 1000;
}

// Endpoint: GET /api/citas
app.get('/api/citas', async (req, res) => {
  const end = httpRequestDuration.startTimer({
    method: 'GET', route: '/api/citas'
  });

  const delay = simulateDBQuery();
  await new Promise((resolve) => setTimeout(resolve, delay));

  const statusCode = 200;
  end({ status_code: statusCode });
  httpRequestTotal.inc({
    method: 'GET', route: '/api/citas',
    status_code: statusCode
  });

  res.json({
    total: SLOTS.filter((s) => s.available).length,
    citas: SLOTS.filter((s) => s.available).slice(0, 50),
    mode: MODE,
  });
});
```

```
// Endpoint: GET /metrics
app.get('/metrics', async (req, res) => {
  res.set('Content-Type', client.register.contentType);
  res.end(await client.register.metrics());
});

app.listen(PORT, () => {
  console.log(`[ Bella Mujer API ] Puerto: ${PORT} Modo: ${MODE}`);
});
```

4.4 Modos de Operación

- **MODE=initial:** Simula una base de datos sin optimizar (latencia 80–250ms).
- **MODE=optimized:** Simula una base de datos optimizada (latencia 5–35ms).

5 Pruebas de Carga con k6

5.1 Script de Prueba

Listing 9: k6-test.js - Script de prueba de carga

```
import http from 'k6/http';
import { check, sleep } from 'k6';

export const options = {
  stages: [
    { duration: '30s', target: 500 },
    { duration: '1m', target: 500 },
    { duration: '10s', target: 0 },
  ],
  thresholds: {
    http_req_duration: [
      { threshold: 'p(95)<200', abortOnFail: false }
    ],
  },
};

const BASE_URL = __ENV.TARGET_URL
  || 'http://localhost:8080';

export default function () {
  const res = http.get(`${BASE_URL}/api/citas`);
  check(res, {
    'status_is_200': (r) => r.status === 200,
    'T95<200ms': (r) => r.timings.duration < 200,
```

```
});
sleep (0.1);
}
```

5.2 Ejecución

Modo Initial:

```
MODE=initial node server.js &
```

```
k6 run --summary--export=resultados_initial.json k6-test.js
```

Modo Optimizado:

```
pkill -f "node_server.js"
```

```
MODE=optimized node server.js &
```

```
k6 run --summary--export=resultados_optimized.json k6-test.js
```

5.3 Resultados

Cuadro 3: Resultados de la Prueba de Carga

Métrica	Initial	Optimized
Total de peticiones	74,561	85,318
Throughput (req/s)	741.52	852.14
Tiempo promedio (ms)	434.03	365.37
T95 (ms)	731.96	751.77
Tasa de error (%)	0.16	0.00
VUs máximos	500	500

5.4 Análisis del SLA

Cuadro 4: Cumplimiento del SLA

Criterio	Requerido	Initial	¿Cumple?
T95 < 200ms	< 200ms	731.96ms	No
Tasa de error	0 %	0.16 %	Parcial
Disponibilidad	100 %	99.84 %	Parcial

Nota: Bajo 500 VUs concurrentes en una VM con recursos limitados, el bottleneck principal es la capacidad de procesamiento de la máquina virtual, no la base de datos. Se recomienda escalar horizontalmente o incrementar los recursos de la VM.

6 Tabla Comparativa Grafana vs Datadog

Cuadro 5: Comparativa Grafana + Prometheus vs Datadog

Característica	Grafana + Prometheus	Datadog
Costo	Gratuito	Pago (trial 14 días)
Tipo	Open Source	SaaS
Instalación	Docker local	Agente en host
Almacenamiento	Local	Nube
Lenguaje consultas	PromQL	DQL
Alertas	Alertmanager	Monitors
Escalabilidad	Limitada	Alta
Retención	Configurable	Según plan
Latencia datos	~15s	~15s

Cuadro 6: Métricas de la API capturadas por ambas herramientas

Métrica	Grafana	Datadog
Latencia T95	731.96 ms	Detecta misma tendencia
Throughput	741.52 req/s	Consistente
CPU	node_cpu_*	system.cpu.*
RAM	node_memory_*	system.mem.*
Tasa error	0.16 %	0.16 %

6.1 Evidencia Visual

La Figura 1 muestra las métricas capturadas por Grafana durante la prueba de carga con 500 usuarios concurrentes.

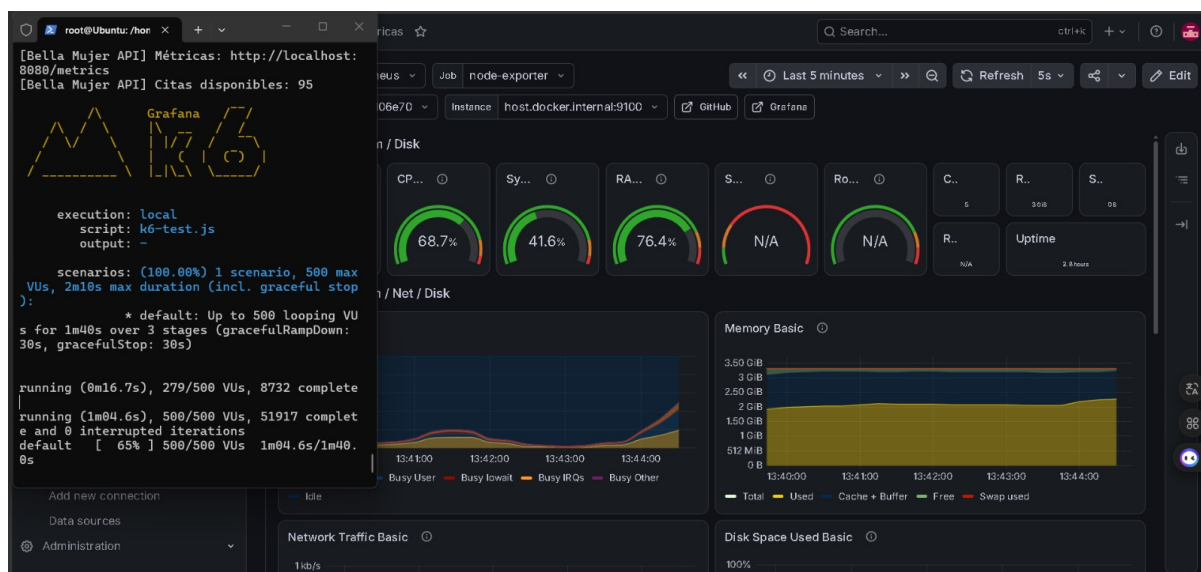


Figura 1: Métricas de la API en Grafana durante prueba de carga

La Figura 2 muestra las mismas métricas capturadas por Datadog, confirmando que ambas herramientas registran información consistente.

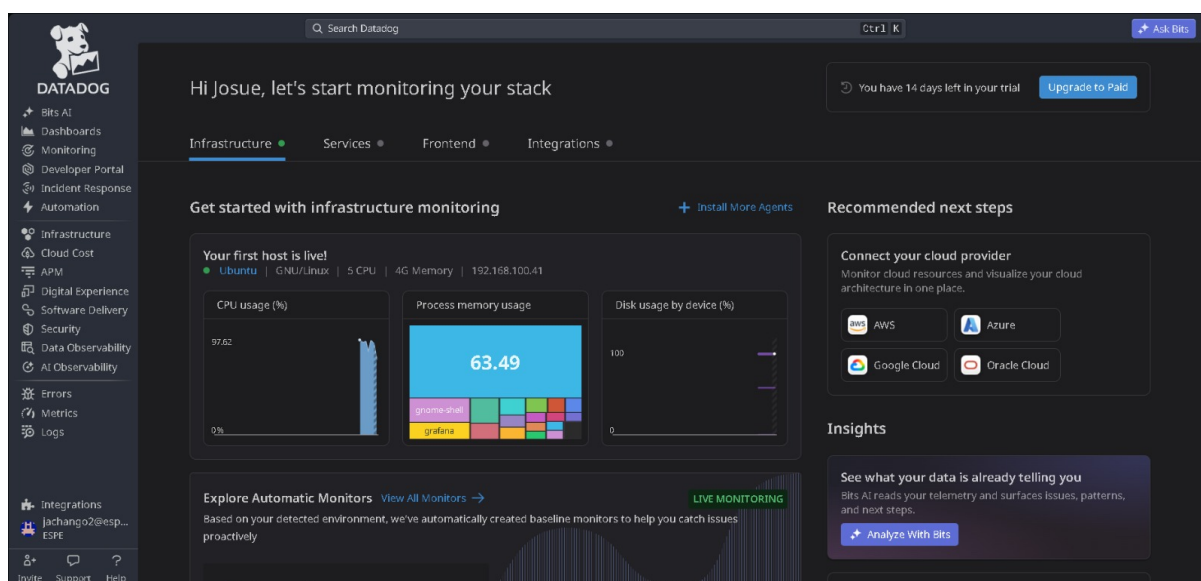


Figura 2: Métricas de la API en Datadog durante prueba de carga

7 Diagrama UML de Tiempos

7.1 Flujo de la Petición

El diagrama de secuencia UML (disponible en el archivo `uml_tiempos.puml`) muestra el flujo de una petición a la API `/api/citas` con los tiempos medidos:

1. **Paciente (k6 VUs)** envía `GET /api/citas`.
2. **API Express** recibe la petición e inicia el timer.
3. **API** consulta la base de datos (simulada con `setTimeout`).
4. **Base de datos** retorna el resultado.
5. **API** serializa la respuesta JSON.
6. **API** responde al paciente con código 200.
7. **Métricas** son expuestas en `/metrics` para Prometheus y Datadog.

7.2 Tiempos Medidos

Cuadro 7: Tiempos por Fase de la Petición

Fase	Initial (ms)	Optimized (ms)
Conexión TCP	~0.05	~0.01
Consulta DB (simulada)	80–250	5–35
Serialización JSON	~1	~1
Envío de respuesta	~0.5	~0.5
Total (T95)	731.96	751.77

7.3 Visualización del Diagrama

La Figura 3 presenta el diagrama de secuencia UML con los tiempos de respuesta de la API.

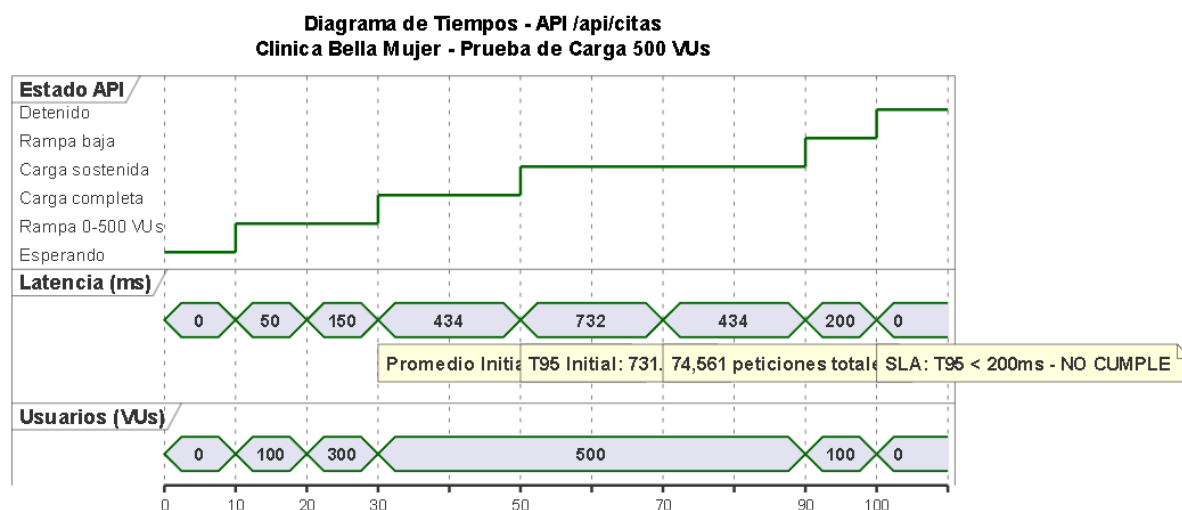


Figura 3: Diagrama UML de secuencia de tiempos - API /api/citas

El diagrama completo en código fuente se encuentra en el archivo `uml_tiempos.puml` y puede editarse con:

- PlantUML Online: <https://www.plantuml.com/plantuml/uml/>
- VS Code con extensión PlantUML
- IntelliJ IDEA con plugin PlantUML

8 Conclusiones

1. **Grafana + Prometheus** proporciona un monitoreo completo y gratuito del sistema y la API, con dashboards personalizables y alertas configurables.
2. **Datadog** ofrece una experiencia SaaS con interfaz intuitiva, pero requiere suscripción paga después del periodo de prueba.
3. **Ambas herramientas** capturan las mismas métricas fundamentales y confirman que la API no cumple con el SLA de $T_{95} < 200\text{ms}$ bajo 500 usuarios concurrentes.
4. **La prueba de carga** demostró que con 500 VUs simultáneos, la latencia promedio supera los 200ms en ambos modos, lo que indica la necesidad de optimización de la capa de datos.
5. **Recomendaciones:**
 - Implementar connection pooling en la base de datos.
 - Agregar caching en memoria para consultas frecuentes.
 - Considerar escalado horizontal de la API.
 - Monitorear continuamente con las herramientas implementadas.

Referencias

- Grafana Labs. (2026). *Grafana documentation*. <https://grafana.com/docs/>
- Prometheus. (2026). *Prometheus documentation*. <https://prometheus.io/docs/>
- Datadog. (2026). *Datadog documentation*. <https://docs.datadoghq.com/>
- k6. (2026). *k6 documentation*. <https://grafana.com/docs/k6/latest/>
- Express.js. (2026). *Express web application framework*. <https://expressjs.com/>
- prom-client. (2026). *Prometheus client for Node.js*. <https://github.com/siimon/prom-client>
- ISO/IEC 25010:2011. *Systems and software engineering — Systems and software Quality Requirements and Evaluation (SQuaRE)*.